**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

# Отчет
# Домашняя работа №3
# Тестирование API средствами Postman

Выполнил:
Люсин Дмитрий
Группа: К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

# Задача

Необходимо реализовать тестирование API средствами Postman, написать тесты внутри Postman и получить один комплексный сценарий по основному процессу приложения.

В качестве предметной области используется приложение для бронирования столиков в ресторанах. Основной пользовательский сценарий должен отражать путь пользователя от регистрации и входа до выбора ресторана, создания бронирования и проверки результата.

# Ход работы

## 1. Подготовка тестовой среды

Для тестирования использовался ранее реализованный REST API на Node.js и Express. Перед запуском тестов сервер поднимается локально, после чего Postman отправляет запросы на адрес приложения.

```
npm run dev
```

Для работы в Postman подготовлены два файла: коллекция запросов и окружение с переменными. Коллекция содержит последовательность запросов, а окружение хранит базовый адрес API и значения, которые передаются между запросами.

| **Файл** | **Назначение** |
|---|---|
| Restaurant_Booking_API_E2E.postman_collection.json | Коллекция Postman с комплексным сценарием и тестами. |
| Restaurant_Booking_Local.postman_environment.json | Окружение Postman с переменными baseUrl, authToken, restaurantId, tableId, bookingId, reviewId. |

## 2. Комплексный сценарий тестирования

В Postman был реализован единый E2E-сценарий, который проверяет основной процесс работы пользователя с приложением. Сценарий состоит из 17 последовательных запросов. Данные, полученные на ранних шагах, автоматически сохраняются в переменные окружения и применяются в следующих запросах.

| **№** | **Метод** | **Действие** | **Endpoint** |
|---|---|---|---|
| 1 | POST | Регистрация нового пользователя | /api/auth/register |
| 2 | POST | Вход пользователя | /api/auth/login |
| 3 | GET | Получение текущего пользователя | /api/auth/me |

| | | | |
|---|---|---|---|
| 4 | GET | Получение списка кухонь | /api/cuisines |
| 5 | GET | Получение списка ресторанов | /api/restaurants |
| 6 | GET | Фильтрация ресторанов | /api/restaurants?... |
| 7 | GET | Просмотр страницы ресторана | /api/restaurants/{restaurantId} |
| 8 | GET | Просмотр меню ресторана | /api/restaurants/{restaurantId}/menu |
| 9 | GET | Просмотр фотографий ресторана | /api/restaurants/{restaurantId}/photos |
| 10 | GET | Просмотр столиков ресторана | /api/restaurants/{restaurantId}/tables |
| 11 | POST | Создание бронирования | /api/bookings |
| 12 | POST | Проверка конфликта бронирования | /api/bookings |
| 13 | GET | История бронирований пользователя | /api/bookings/my |
| 14 | GET | Просмотр созданного бронирования | /api/bookings/{bookingId} |
| 15 | POST | Создание отзыва по бронированию | /api/restaurants/{restaurantId}/reviews |
| 16 | GET | Проверка отзыва в списке отзывов | /api/restaurants/{restaurantId}/reviews |
| 17 | PATCH | Отмена бронирования | /api/bookings/{bookingId}/cancel |

## 3. Переменные окружения Postman

Для передачи данных между запросами использованы переменные окружения. Это позволяет не вводить вручную JWT-токен и идентификаторы сущностей. Например, после

входа сохраняется authToken, после выбора ресторана - restaurantId, после создания бронирования - bookingId.

| Переменная | Назначение |
|---|---|
| baseUrl | Базовый адрес локального API. |
| authToken | JWT-токен авторизованного пользователя. |
| cuisineId | ID кухни для фильтрации ресторанов. |
| restaurantId | ID выбранного ресторана. |
| tableId | ID выбранного столика. |
| bookingId | ID созданного бронирования. |
| reviewId | ID созданного отзыва. |

## 4. Тесты внутри Postman

Внутри запросов были написаны тесты на JavaScript с использованием объекта pm. Тесты проверяют HTTP-статусы, структуру JSON-ответов, наличие обязательных полей и корректность бизнес-логики.

Всего в коллекции содержится 17 запросов и 47 проверок. Это позволяет автоматически убедиться, что основной сценарий приложения выполняется корректно.

```
pm.test("Статус ответа 200", function () {
    pm.response.to.have.status(200);
});

const json = pm.response.json();
pm.environment.set("authToken", json.token);
```

Отдельно проверяется ситуация конфликта бронирования. В сценарии повторно отправляется запрос на бронирование того же столика на то же время. Корректным результатом является ответ 409 Conflict, так как система не должна допускать двойное бронирование одного столика.

## 5. Проверяемые результаты

В результате выполнения коллекции проверяется, что пользователь может зарегистрироваться, войти в систему, получить список ресторанов, применить фильтры, открыть страницу ресторана, посмотреть меню, фотографии и столики, создать бронирование, увидеть его в истории, оставить отзыв и отменить бронирование.

При успешном выполнении Collection Runner показывает прохождение тестов. Ошибка 409 в запросе проверки конфликта является ожидаемым результатом и подтверждает работу бизнес-правила о запрете повторного бронирования.

## Вывод

В ходе работы было реализовано тестирование REST API средствами Postman. Была создана коллекция с комплексным пользовательским сценарием и окружение с переменными для передачи данных между запросами.

Сценарий покрывает основной процесс приложения бронирования столиков: регистрацию, вход, поиск и фильтрацию ресторанов, просмотр ресторана, создание бронирования, просмотр истории, создание отзыва и отмену бронирования. Внутри запросов написаны автоматические проверки статусов, структуры ответов и ключевых бизнес-правил.

Таким образом, API было проверено не отдельными ручными запросами, а полноценным последовательным сценарием, который показывает работоспособность основной логики приложения.